Vanity Palms - Product Handoff

Vanity Palms - Product Handoff Document

Version: 1.0 **Date:** April 6, 2026 **Live URL:** https://vanity-palms.pages.dev **Repository:** conductor-playground/tyler

---

Executive Summary

Vanity Palms is a full-stack aesthetic provider discovery platform for Los Angeles. Patients discover verified cosmetic surgeons, visualize potential outcomes with AI, compare providers side-by-side, and request consultations. Providers manage profiles, consultations, galleries, and analytics through a dedicated dashboard. Admins oversee provider verification, subscriptions, and platform content.

Tech Stack: React 18 + TypeScript + Vite + Tailwind CSS + shadcn/ui + Supabase (Postgres, Auth, Storage, Edge Functions) + MediaPipe + Gemini AI

The 5-Step User Journey

The platform follows a structured patient flow:

1. **Choose Concern** - Patient selects an aesthetic concern (Face, Nose, Skin, Body, etc.) from the homepage
2. **Explore Treatments** - Browse surgical and non-surgical procedures for that concern with cost, recovery, and rating data
3. **Visualize** - Upload a photo to the AI Visualizer to preview potential outcomes before committing
4. **Compare** - Select up to 3 providers and compare them side-by-side on pricing, credentials, experience, and reviews
5. **Book** - Request a consultation directly from any provider's profile or comparison page

---

Pages & Features

Homepage (/)

The landing page introduces Vanity Palms with a hero section, platform stats (500+ providers, 50k+ consultations), the "How It Works" 5-step journey, a dynamic "Browse by Concern" grid, and a "Why Different" feature section. All sections use scroll-reveal animations and smooth hash-based navigation.

Provider Listing (/providers)

Full-featured provider directory with search, advanced filtering (procedure, distance, rating, price range, experience, gender, consultation type), sorting (rating, distance, reviews, experience), and multi-select comparison (up to 3 providers). Provider cards show photo, name, specialty, rating, distance, response time, and verification status.

Provider Profile (/providers/:id)

Detailed provider page showing credentials, specialty tags, certification badges, procedures offered with pricing, a review section with metrics breakdown, a before/after gallery from Supabase storage, Instagram feed integration, and a prominent “Request Consultation” button. Tracks profile views via analytics.

Compare Providers (/compare)

Side-by-side comparison table for 2-3 providers. Compares rating, reviews, location, distance, experience, response time, consultation types, certifications, and per-procedure pricing. Each provider has a “Request Consultation” button. A dark CTA banner at the bottom encourages booking.

Procedure Detail (/procedures/:id)

Educational page for each procedure: overview, how it works, recovery timeline (day-by-day), expected results, patient reviews, cost range, and providers who offer it. Links back to concern and forward to provider listings filtered by procedure.

Concern Detail (/concerns/:id)

Concern-specific landing page listing all related procedures (surgical and non-surgical) with tabs for filtering. Includes a CTA to try the AI Visualizer. Procedures sorted by popularity with cost, recovery, and rating data.

AI Visualizer (/visualizer)

Interactive face-editing tool powered by MediaPipe (468-point face mesh) and Gemini AI image generation. Features include:

- **Photo Upload Wizard** - Guided upload with photo quality tips
- **Feature Sliders** - Nose (width, length, bridge), Eyes (size, brow lift), Lips (fullness, width), Jawline (sharpness), Cheeks (fullness), Skin (texture, tone)
- **Chat Interface** - Natural language instructions (“make my nose thinner”) parsed into feature adjustments
- **AI Generation** - Photorealistic results via Gemini 2.0 Flash with FLUX.1 Kontext fallback
- **Before/After Toggle** - Compare original and modified images
- **Undo/Redo History** - Full edit history tracking
- **Download** - Save result images
- **“Find Providers” CTA** - Bridge to provider discovery after visualization

Consultation Chat (/consultation/:token)

Token-gated message thread between patient and provider. Patients access via a unique token (no login required). Providers access through their dashboard. Real-time messaging via Supabase Realtime subscriptions. Shows scheduled meeting details when booked (date, mode, location, notes).

Review Request (/review/:token)

Multi-step review form with stage-specific questions: - **Consultation Stage**: consultation experience, office environment, doctor listening, pressure assessment - **Procedure Stage**: results satisfaction, recovery experience, post-op timeline, complications - **Results Stage**: long-term satisfaction, natural appearance, expectations match

Uses star ratings, boolean yes/no toggles, and text input. Accessible ARIA attributes on all interactive elements.

Location/Procedure Pages (/procedures/:slug/:location)

SEO-optimized pages for procedure+location combos (e.g., “rhinoplasty in Los Angeles”). Shows local providers, procedure education, and local reviews.

User Dashboard (/account)

Patient account page showing consultation history with status badges, quick actions (browse providers, try visualizer), and account details. Links to active consultation threads.

Provider Dashboard (/provider/dashboard)

Comprehensive provider workspace with 7 tabs:

- **Dashboard**: Quick stats (views, consultations, messages), recent activity
- **Inbox**: Consultation requests with filtering, message previews, scheduling, review request generation
- **Profile**: Full profile editor (name, bio, specialties, certifications, languages, photo, location, consultation types, credentials verification with document upload)
- **Analytics**: Profile views, consultation trends, conversion metrics, event breakdown charts
- **Subscription**: Tier comparison (free vs premium), feature breakdown
- **Gallery**: Upload/manage before-after images stored in Supabase storage
- **Settings**: Password change, notification preferences, Composio integration status, account deactivation

Provider states: Pending (awaiting admin approval with support contact), Approved (full dashboard access), Rejected (with support contact and appeal info).

Admin Console (/admin)

Platform management with tabs for: - **Overview**: Provider count, consultation volume, platform health - **Providers**: Provider list with search, status management (approve/reject), subscription tier adjustment - **Content**: Review moderation, content flagging - **Subscriptions**: Billing management, manual tier changes - **Whitelist**: Email whitelist management (add/remove provider emails)

Auth Pages

- **User Login/Signup** (/login, /signup): Email/password + Google OAuth
- **Provider Login/Signup** (/provider/login, /provider/signup): Email/password + Google OAuth, whitelist validation
- **Provider Password Reset** (/provider/reset-password): Token-based password reset flow

Static Pages

- **About** (/about): Platform mission and audience
- **Privacy** (/privacy): Data collection and usage policies
- **Terms** (/terms): Platform usage expectations
- **Cookies** (/cookies): Cookie and analytics usage

AI Visualizer Pipeline (Technical)

The visualization system has 5 components:

1. **Face Mesh Detection** (useFaceMesh.ts) - MediaPipe FaceLandmarker detects 468 facial landmarks from uploaded photos. Runs on GPU with CPU fallback via WASM.
2. **Landmark Mapping** (landmarks.ts) - Maps landmark indices to facial features (nose, eyes, lips, jaw, cheeks, skin). Defines transformation calculations with bilateral symmetry.
3. **Mesh Warping** (warp.ts) - Moving Least Squares algorithm applies landmark deltas to the image. Grid-optimized for performance with bilinear interpolation.
4. **Chat Parser** (chatParser.ts) - NLP engine converts natural language (“make my nose thinner”) into feature deltas. Supports 60+ keyword patterns with intensity modifiers.

5. **AI Pipeline** (aiPipeline.ts) - Calls Supabase Edge Function which routes to Gemini 2.0 Flash (primary) or FLUX.1 Kontext (fallback) for photorealistic image generation.

Database Schema

Tables

Table	Purpose	Key Fields
concerns	Aesthetic concern categories	name, icon, description, procedure_count
procedures	Treatment definitions	name, type, cost_min/max, recovery_days, rating, overview
providers	Provider profiles	name, specialty[], rating, certifications[], location, subscription_tier
provider_procedures	Provider-procedure pricing	provider_id, procedure_id, price
consultations	Consultation requests	access_token, provider_id, patient_name, email, status, scheduled_at
consultation_messages	Chat messages	consultation_id, sender_type, body, read_at
reviews	Patient reviews	provider_id, procedure_id, rating, body, stage, structured_answers
review_requests	Review invitation tokens	token, consultation_id, procedure_id, stage, expires_at
analytics_events	Tracking events	provider_id, event_type, metadata
admins	Admin users	user_id, email
provider_whitelist	Signup whitelist	email, note, invited_by

RPC Functions

Function	Purpose
is_admin()	Check if current user is admin
is_approved_provider()	Check if provider is approved
is_whitelisted_email()	Check provider signup eligibility
ensure_admin_if_designated()	Auto-grant admin for designated emails
claim_provider_by_email()	Link auth user to existing provider record
create_review_request()	Generate review token for a consultation
get_review_request()	Fetch review request details by token
submit_review_request()	Submit review from token (6 rating fields + structured answers)
get_public_consultation()	Token-based consultation lookup
get_public_consultation_messages()	Token-based message retrieval

Function	Purpose
send_public_consultation_message()	Token-based message sending (5000 char limit)
get_unread_message_count()	Provider unread message count
admin_update_provider_status()	Admin: change provider status
admin_update_provider_subscription()	Admin: change subscription tier
admin_delete_review()	Admin: remove a review
admin_add/remove_provider_whitelist()	Admin: manage whitelist

Row Level Security

All tables use RLS. Key patterns: - **Public read** for approved providers, procedures, concerns, reviews - **Token-gated access** for consultations and messages (patients use access_token, not auth) - **Provider-scoped write** for own profile, consultations, messages - **Admin override** via `is_admin()` wrapped in `(SELECT ...)` for per-statement evaluation - **Whitelist-gated signup** for providers

---

Security Measures

- **JWT verification** enabled on Edge Functions
- **Email confirmations** required for signup
- **Password policy**: 12+ characters with uppercase, lowercase, digits, and symbols
- **CORS**: Edge function restricted to `ALLOWED_ORIGINS` (no wildcard)
- **CSP headers**: Full Content-Security-Policy on Cloudflare Pages
- **X-Frame-Options**: DENY (clickjacking protection)
- **MIME validation**: Image uploads validated for type and size (10MB max)
- **RLS policies**: All database tables protected with row-level security
- **Security Definer functions**: All RPCs use `SECURITY DEFINER` with `SET search_path = public`
- **Input validation**: Edge function validates image size (10MB), prompt length (2000 chars), MIME whitelist
- **No hardcoded secrets**: All API keys in environment variables/Supabase secrets
- **FK indexes**: 10 foreign key indexes for query performance

---

Accessibility

- **Skip link** for keyboard navigation (skip to main content)
- **ARIA attributes** on star ratings (`role="radiogroup"` , `aria-checked`) and boolean toggles
- **No nested interactive elements** in navigation (all `<Link><button>` patterns removed)
- **44px touch targets** on slider thumbs for mobile accessibility
- **Scroll-reveal animations** respect `prefers-reduced-motion`
- **Semantic HTML** with proper heading hierarchy

Performance

- **Code splitting**: React.lazy + Suspense for all 17 routes

- **React.memo**: Applied to list-rendered components (ProviderCard)
- **Async font loading**: Google Fonts loaded via async `<link>` with `preconnect` (non-render-blocking)
- **Image optimization**: Provider images served from Supabase Storage with cache headers
- **Query optimization**: N+1 queries eliminated (fetchProvidersByProcedure uses single JOIN)
- **Test coverage**: 22 tests across 7 files with coverage thresholds configured

---

Deployment

- **Hosting**: Cloudflare Pages (https://vanity-palms.pages.dev)
- **SPA Routing**: `_redirects` file with `/* /index.html 200`
- **Security Headers**: `_headers` file with CSP, X-Frame-Options, etc.
- **Backend**: Supabase hosted (nzcirreefefdzabnzeeh.supabase.co)
- **Edge Functions**: Supabase Edge Functions (Deno runtime)
- **Storage**: Supabase Storage (provider-images bucket, 5MB limit, PNG/JPEG/WebP)

Admin Access

Email	Role	Notes
aalang@ucsc.edu	Admin + Provider	Founding admin, Dr. Aiden Lang provider profile
Kevinramirez889@gmail.com	Admin	Password: TestPassword1!

Admin console accessible at /admin after login.

Environment Variables

Variable	Purpose
VITE_SUPABASE_URL	Supabase project URL
VITE_SUPABASE_ANON_KEY	Supabase anonymous key
GEMINI_API_KEY	Google AI Studio key (Supabase secret)
ALLOWED_ORIGINS	CORS whitelist for edge function (Supabase secret)

Remaining Work

Must Do

- Split ProviderDashboard.tsx (2215 lines) into sub-components
- Split api.ts (1147 lines) into domain modules
- Adopt TanStack Query for data fetching (already installed)
- Enable strictNullChecks in TypeScript config

Should Do

- Move analytics aggregation server-side
- Add Zod schemas for API response validation
- Add requestAnimationFrame throttle to warp rendering
- Increase toolbar button size for mobile in Visualizer

Nice to Have

- Sentry error reporting integration
- Offline detection
- Move MLS warp to Web Worker
- Host MediaPipe WASM locally (remove CDN dependency)
- Write tests for api.ts, warp.ts, and auth flows